

Lab: Footprinting and Reconnaissance

OSINT Techniques — Google Hacking, Website & Archive Footprinting, Content Extraction, Email Header Tracing, and DNS Enumeration

Course: Hands-On Ethical Hacking and Network Defense — Footprinting & Reconnaissance module companion lab

Estimated time: 2–2.5 hours **Format:** Individual or paired; requires internet access (see Section 3) **Prerequisite:** Comfort with a Linux terminal is helpful but not required — every tool here runs equally well from your host OS.

1. Learning Objectives

By the end of this session you will be able to:

- Use Google’s advanced search operators (“Google hacking” / “Google dorking”) to surface information that isn’t meant to be easily discoverable
- Extract ownership, hosting, and technology-stack information from a live website
- Recover historical content from a site using the Wayback Machine
- Mirror a website locally for offline analysis
- Trace the path an email actually took by reading its headers
- Enumerate a domain’s DNS footprint (A, MX, NS, TXT, SOA records) and test for a misconfigured zone transfer

2. ⚠ Scope and Target Selection — Read This First

Unlike a lab run entirely inside an isolated VM, most of this lab reaches out to the real internet — so target selection is the whole ballgame.

Rules for every part below:

- **Use** `example.com` for any task that queries a live server directly (DNS lookups, zone-transfer attempts, WHOIS) — it’s reserved by the IANA specifically for use in documentation and examples like this one.
- **Use a domain you personally own** (a portfolio site, a personal blog) wherever you want a more realistic, personal target — you already have standing authorization, because it’s yours.
- **Never** run these techniques — especially the website-mirroring task — against a real company’s live site without their written permission. A search-engine query touches Google’s index, not the target; a full site crawl touches the target directly and repeatedly, and can look like an attack from their side.
- For the mirroring task specifically, use **books.toscrape.com** — a public sandbox site built for exactly this kind of practice — unless your instructor assigns a different, explicitly authorized target.

If your instructor has assigned a specific practice domain for this module, use that instead of `example.com` wherever it fits.

3. Environment Note — This Lab Needs Internet Access

If you’ve been working through an isolated attack lab in earlier sessions, that VM was deliberately cut off from the internet — that isolation is correct for attacking a local vulnerable target, and you should leave it that way. For this lab, use one of:

- Your host operating system directly (every tool below runs fine there), or
- A **separate** internet-connected Kali/Linux instance — a second VM with a NAT or bridged adapter, or Kali running on real hardware

Do not add a NAT or bridged adapter to an isolated attack VM to do this lab — its entire purpose is staying isolated, and reconnecting it defeats the point of building it that way.

4. Tools Used This Session

- A web browser
- `whois`, `dig` / `nslookup` / `host` (built into most OSes; pre-installed on Kali)
- `dnsrecon` or `dnsenum` (Kali; optional but recommended)
- `whatweb` (Kali; optional)
- HTTrack (`httrack`, pre-installed on Kali; also available for Windows/macOS)
- Your own email account (any provider)

5. Procedure

Part A — Google Hacking

1. Read through the operator reference below before searching:

Operator	What it finds
<code>site:</code>	Results only from one domain
<code>filetype:</code>	Only files of a given type (pdf, xls, docx...)
<code>intitle:</code>	Pages with a word in the <code><title></code>
<code>inurl:</code>	Pages with a word in the URL
<code>cache:</code>	Google's last cached copy of a page
<code>link:</code>	Pages linking to a target URL (limited support on modern Google)

2. Run at least **six** searches combining these operators against your chosen target (Section 2), for example:

```
site:example.com filetype:pdf
site:example.com inurl:admin
intitle:"index of" site:example.com
```

3. For each search, record: the query, the number of results, and whether anything you found looks like it shouldn't be public (a spreadsheet, a login page, a directory listing).
4. Skim the **Google Hacking Database** (exploit-db.com/google-hacking-database) for five minutes. Pick two categories relevant to your target's type of site (e.g., "Files containing passwords," "Sensitive directories") and note what each looks for.

Part B — Website Information Footprinting

1. Run WHOIS against your target's registered domain:

```
whois example.com
```

Record: registrar, registration/expiry dates, registrant organization (if not privacy-shielded), name servers.

2. Pull a **Netcraft Site Report** (netcraft.com/site-report — search your target's domain) and record: hosting provider, IP address, and the site's first-seen date if shown.
3. View your target's page source (Ctrl+U / "View Page Source") and note: the CMS or framework in use (look for generator meta tags, common file paths like `/wp-content/`), and any comments left in the HTML.
4. If available, run `whatweb` against the target from a terminal:

```
whatweb example.com
```

Record every technology it reports.

Part C — Archived Website Information

1. Go to the **Wayback Machine** (web.archive.org) and search your target domain.
2. Find the **earliest** archived snapshot and the **most recent** one. Compare: what changed? A redesign, different contact info, a page that no longer exists?
3. Look specifically for a page type that often survives in archives long after it's removed live — a staff directory, an old contact page, an internal tool login page. Record whether you found one.
4. **Reflection (1-2 sentences):** Why would an old, "deleted" page still matter to someone footprinting a target today?

Part D — Extracting Website Contents (Mirroring)

1. On your target (books.toscrape.com, unless assigned otherwise), run HTTrack:

```
httrack "http://books.toscrape.com" -O ./mirror -%v
```

2. Let it finish, then browse the mirrored copy **offline** (disconnect from Wi-Fi, or just open the local `index.html` directly) and confirm the site still mostly works.

3. Search the mirrored files on disk for anything interesting:

```
grep -ril "email" ./mirror
grep -ril "password" ./mirror
```

4. Record: how many pages/files did HTTrack pull down, and did anything appear in your `grep` search that wouldn't have been obvious from browsing the live site normally?

Part E — Tracing a Received Email

1. Send yourself a test email from a **different** account or provider (a friend's address, or a second account of your own) so the header has a real path to trace.
2. Open the email and view its full headers — in Gmail: **⋮ → Show original**; in Outlook: **File → Properties → Internet headers**. Copy the full header text.
3. Find the chain of `Received:` lines. Each one is a hop the message passed through, oldest at the bottom. For at least three hops, record the reported hostname and IP address.
4. Run `whois` on the **originating** IP address (the hop closest to the sender) and record who owns that IP block.
5. Cross-check the sender's claimed domain (from the `From:` header) against the `Received:` chain and the `SPF / DKIM` results near the top of the headers, if present. Do they agree?

Reflection (2-3 sentences): If the `From:` address and the earliest `Received:` hop don't match up, what does that suggest — and why is this exactly the kind of check that catches spoofed email?

Part F — DNS Footprinting

1. Pull the core record types for your target:

```
dig example.com A
dig example.com MX
dig example.com NS
dig example.com TXT
dig example.com SOA
```

Record each result. Note anything in the TXT records (SPF, DKIM, and verification strings often live here).

2. Attempt a zone transfer against each name server you found in Step 1:

```
dig axfr @<nameserver> example.com
```

This will almost certainly be refused — that's correct and expected. **If it ever succeeds against a domain you don't own, stop, do not save or use the data, and report it to your instructor** — a working zone transfer is a real misconfiguration, not a lab exercise.

3. If installed, run a DNS enumeration tool for a fuller picture:

```
dnsenum example.com
```

or

```
dnsrecon -d example.com
```

Record any subdomains it finds that you hadn't already seen.

6. Bringing It Together

Build a single footprinting summary table for your target, pulling one row from **each** part above (six rows total): technique, key finding, and why it matters to someone assessing this target's exposure.

7. What to Submit

1. Part A: six Google-dork queries and results, plus your two GHDB category notes
2. Part B: WHOIS, Netcraft, page-source, and WhatWeb findings
3. Part C: earliest/latest snapshot comparison, plus the reflection answer
4. Part D: HTTrack summary and `grep` results
5. Part E: your header trace (three hops) and the reflection answer — **redact the actual email address and any personal content**, keep only the technical header fields

6. Part F: your DNS record dump, zone-transfer attempt result, and enumeration tool output
7. Your Section 6 summary table

8. Troubleshooting

Symptom	Likely cause / fix
Google returns almost nothing for a <code>site:</code> search	Small/personal sites are sometimes lightly indexed — fall back to a larger practice domain your instructor assigns
<code>whatweb</code> or <code>htttrack</code> not found	Install with <code>sudo apt install whatweb htttrack</code> on Kali/Debian-based systems
Zone transfer command hangs	Normal — the server usually just drops the connection rather than replying; Ctrl+C after a few seconds and move on
Email headers show no useful <code>Received:</code> chain	Some webmail providers compress the chain — try downloading the email as <code>.eml</code> and opening it in a plain text editor instead

This lab is a standalone Footprinting & Reconnaissance session — slot it into whichever point in your course covers this module.